

TESTDOKUMENTATION & QUALITÄTSSICHERUNGSBERICHT

Smart Restaurant – Digitales Bestell- und Verwaltungssystem

Ausbildungsberuf: Fachinformatiker/in für Anwendungsentwicklung

Projektbezeichnung:	Smart Restaurant (LF10-Projekt)
Auftragnehmer / Team:	Another Great Solution GmbH (Team „Der Dreier“)
Auftraggeber (Szenario):	Regionales Gastronomieunternehmen
Programmiersprache & DB:	Java 17 (OpenJDK) & SQLite 3 über JDBC
Dokumentenstand & Version:	01. Oktober 2026 Version 1.2 (Final)
Gesamtergebnis QS:	100 % Tests Bestanden (16/16 Systemtests, 5/5 Automatisierte Tests)

1. Einleitung, Testziel & Testphilosophie

Die vorliegende Testdokumentation dient der systematischen Verifikation und Validierung der Software **Smart Restaurant Admin**. Gemäß den Richtlinien für IHK-Abschlussprojekte und dem Lehrplan des Lernfelds 10 stellt dieses Dokument sicher, dass die implementierte Lösung sämtliche fachlichen Anforderungen erfüllt, robust gegenüber Fehleingaben ist und die Datenintegrität zu jeder Zeit gewahrt bleibt.

2. Teststrategie & Teststufen (Testpyramide)

Um ein optimales Verhältnis zwischen Testtiefe und Ausführungsgeschwindigkeit zu erzielen, folgt das Qualitätssicherungskonzept der agilen Testpyramide nach Mike Cohn:

- **Unit-Tests:** Prüfung isolierter Hilfsmethoden, Datenmodelle und Preisvalidierungen.
- **Integrationstests (Automatisierte DAO-Suite):** Automatisierte Überprüfung der JDBC-Interaktion zwischen Java-Entitäten und der SQLite-Datenbank über die Klasse `DatabaseTest.java`.
- **System- & GUI-Tests:** Strukturierte manuelle Verifikation aller Dialogmasken, Tabellen, Menüs und Eingabewizards.
- **End-to-End- & Akzeptanztests (UAT):** Prüfung zusammenhängender Betriebsabläufe (vom Tisch belegen über die Zubereitung bis zum Bezahlen).

3. Testumgebung & Testmittel

Parameter	Spezifikation der Testumgebung
Betriebssysteme	Microsoft Windows 11 Pro (64-Bit) / Referenzprüfung unter Ubuntu Linux 22.04 LTS
Java-Laufzeitumgebung	OpenJDK Version 17.0.10 (Eclipse Temurin LTS)
Datenbank & Treiber	SQLite Version 3.42+ embedded, Xerial SQLite-JDBC 3.x (<code>lib/sqlite-jdbc.jar</code>)
Web-Dashboard Server	Eingebetteter Multi-Threaded Java HTTP-Server (<code>webServer.java</code>) auf Port 8080
Hardware-Ausstattung	Intel Core i7 / AMD Ryzen 7, 16 GB RAM, FHD Display (1920 × 1080)

4. Detaillierter Testkatalog (System- & Oberflächenprüfung)

Die folgende Testmatrix dokumentiert die systematische Durchführung aller 16 Systemtestfälle für die fünf Kernmodule der Anwendung.

ID	Testobjekt & Ziel	Eingaben & Testschritte	Soll-Ergebnis	Status	Tester
T-SYS-01	DB-Schema Initialisierung	Programmstart bei gelöschter <code>smart_restaurant.db</code> .	DB-Datei wird angelegt, DDL-Befehle erzeugen 4 Tabellen, Standard-Stammdaten werden befüllt.	PASS	Team Dreier

ID	Testobjekt & Ziel	Eingaben & Testschritte	Soll-Ergebnis	Status	Tester
T-SYS-02	Web-Dashboard Start	Programmstart; Aufruf von <code>http://localhost:8080</code> .	Integrierter HTTP-Server antwortet mit Status 200 und rendert das mobile Dashboard.	PASS	Team Dreier
T-GUI-01	Mitarbeiter anlegen	Name: „Maximilian Weber“, User: „mweber“, Rolle: „Service“ -> Klick „Hinzufügen“.	Datensatz in SQLite gespeichert; JTable zeigt neuen Mitarbeiter sofort an.	PASS	Team Dreier
T-GUI-02	Mitarbeiter Validierung Leerfeld	Felder leer lassen -> Klick „Hinzufügen“.	Warnungs-Dialog: „Bitte alle Felder ausfüllen!“ erscheint; kein DB-Insert.	PASS	Team Dreier
T-GUI-03	Mitarbeiter Unique User	Bereits existierenden Benutzernamen „mweber“ erneut erfassen.	Fehlerhinweis zu doppeltem Benutzernamen; Datenbank-Constraint schützt Datenbestand.	PASS	Team Dreier
T-GUI-04	Mitarbeiter löschen	Mitarbeiter in JTable selektieren -> Klick „Löschen“.	Datensatz wird aus SQLite entfernt; Zeile verschwindet aus Tabelle.	PASS	Team Dreier
T-GUI-05	Artikel Preis mit Komma	Name: „Lasagne“, Kat: „Hauptspeisen“, Preis: „12,50“ -> Klick „Hinzufügen“.	Deutsches Komma wird automatisch in Dezimalpunkt gewandelt; 12.50 € gespeichert.	PASS	Team Dreier
T-GUI-06	Artikel Preis Validierung	Preis: „ungültig“ eingegeben -> Klick „Hinzufügen“.	Hinweisdialog: „Gültigen Preis eingeben!“ erscheint; keine unbehandelte Exception.	PASS	Team Dreier
T-GUI-07	Artikel Negativer Preis	Preis: „-4.50“ eingegeben -> Klick „Hinzufügen“.	Betrag wird abgewiesen (Preis muss > 0 sein); kein fehlerhafter Eintrag.	PASS	Team Dreier
T-GUI-08	Artikel Wizard Dialog	Klick auf „Neuer Artikel (Wizard)“ -> Maske ausfüllen und bestätigen.	Modaler Wizard führt Anwender durch Erfassung und übernimmt Daten in JTable.	PASS	Team Dreier

ID	Testobjekt & Ziel	Eingaben & Testschritte	Soll-Ergebnis	Status	Tester
T-GUI-09	Tisch anlegen	Tischnummer: 15, Kapazität: 6, Status: „frei“ -> Klick „Hinzufügen“.	Tisch wird in SQLite gespeichert und in Tischliste aufgeführt.	PASS	Team Dreier
T-GUI-10	Tisch Duplikat-Nummer	Bereits vergebene Tischnummer 15 erneut eingeben.	Unique-Constraint greift: Fehlermeldung informiert Anwender; kein Crash.	PASS	Team Dreier
T-GUI-11	Tisch Kapazitätsprüfung	Kapazität: „0“ oder ungültiger Text.	Meldung: Kapazität muss mindestens 1 Sitzplatz betragen.	PASS	Team Dreier
T-GUI-12	Bestellstatus Übergang	Bestellung mit Status „aufgegeben“ markieren -> Klick „Status weiterschalten“.	Status wechselt deterministisch auf „in Bearbeitung“ in UI und SQLite.	PASS	Team Dreier
T-GUI-13	Bestellstatus Vollzyklus	Mehrfach „Status weiterschalten“ betätigen bis Status „bezahlt“.	Kette: aufgegeben → in Bearbeitung → fertig → serviert → bezahlt sauber durchlaufen.	PASS	Team Dreier
T-GUI-14	Bestellübersicht SQL-JOIN	Reiter „Bestellübersicht“ öffnen.	Tischnummer und Mitarbeitername werden über Foreign-Key-JOIN aufgelöst dargestellt.	PASS	Team Dreier
T-GUI-15	System- & DB-Status	Reiter „System & DB Status“ öffnen.	Statusanzeige zeigt grüne Indikatoren, DB-Dateipfad und Treiber-Version.	PASS	Team Dreier
T-GUI-16	Manuelle Tabellen-Aktualisierung	Klick auf „Aktualisieren“ in beliebigen Reitern.	Tabelle lädt aktuellen Datenbestand flackerfrei neu aus SQLite.	PASS	Team Dreier

5. Automatisierte Testsuite (Integrationstests & DAO)

Zur Absicherung von Regressionen bei Quellcodeänderungen wurde die automatisierte Testsuite `com.smartrestaurant.DatabaseTest` implementiert. Sie prüft sämtliche CRUD-Operationen direkt gegen die SQLite-Engine.

Ausführungsprotokoll des Test-Runners:

```
=== AUTOMATISIERTER TEST: DatabaseManager & SQL ===
[PASS] 1. Datenbank erfolgreich initialisiert.
  ✓ Mitarbeiter konnte hinzugefügt werden.
  ✓ Mitarbeiter-Anzahl hat sich um 1 erhöht.
  ✓ Mitarbeiter konnte gelöscht werden.
[PASS] 2. Mitarbeiter CRUD-Operationen erfolgreich.
  ✓ Artikel konnte hinzugefügt werden.
  ✓ Artikel-Anzahl hat sich um 1 erhöht.
  ✓ Artikel konnte gelöscht werden.
[PASS] 3. Artikel CRUD-Operationen erfolgreich.
  ✓ Tisch konnte hinzugefügt werden.
  ✓ Tisch-Anzahl hat sich um 1 erhöht.
  ✓ Tisch konnte gelöscht werden.
[PASS] 4. Tisch CRUD-Operationen erfolgreich.
  ✓ Bestellungen vorhanden.
  ✓ Bestellstatus konnte auf 'in Bearbeitung' aktualisiert werden.
[PASS] 5. Bestellstatus-Update erfolgreich.

☑ ALLE AUTOMATISIERTEN TESTS ERFOLGREICH BESTANDEN!
Laufzeit: 38 ms | Ausgeführte Assertions: 12 | Fehler: 0 | Fehlgeschlagen: 0
```

6. Sicherheitstests & Grenzwertanalysen

- **SQL-Injection-Prüfung:** Eingabe von böartigen Payloads wie `' OR '1'='1` und `'; DROP TABLE bestellung; --` in alle Textfelder. Ergebnis: Durch die konsequente Verwendung von `PreparedStatement` mit gebundenen Parametern werden alle Sonderzeichen ausnahmslos als Literale verarbeitet. Kein Angriffspunkt vorhanden.
- **Numerische Grenzwerte:** Validierung von Mindestpreisen (0,01 €), Höchstbeträgen und Ganzzahlprüfungen bei Tischnummern.
- **Zeichenkettenlängen:** Stresstests mit überlangen Zeichenfolgen (250+ Zeichen) zeigten keine Speicherlecks oder grafischen Verzerrungen im GUI-Layout.

7. Fehlermanagement & Abweichungsbericht

Bug-ID	Fehlerbeschreibung	Schweregrad	Korrekturmaßnahme	Status
BUG-01	Komma in Preisfeldern verursachte <code>NumberFormatException</code> .	Mittel	String-Ersetzung <code>.replace(',', ' ')</code> vor dem Double-Parsing implementiert.	Behoben
BUG-02	<code>JTable</code> -Zeilenselektion blieb nach Löschen auf ungültigem Index.	Gering	Aufruf von <code>clearSelection()</code> im Aktualisierungsablauf integriert.	Behoben

Bug-ID	Fehlerbeschreibung	Schweregrad	Korrekturmaßnahme	Status
BUG-03	Statusweitschaltung war auch im Zustand „bezahlt“ aktiv.	Gering	Endzustandsprüfung implementiert; abgeschlossene Bestellungen bleiben unverändert.	Behoben

8. Testfazit & Freigabeempfehlung

Gesamtbewertung der Qualitätssicherung:

Sämtliche 16 manuellen Systemtests sowie alle 5 automatisierten Testblöcke wurden mit dem Ergebnis **PASS (100 %)** absolviert. Die Software erweist sich als fehlerfrei, benutzerfreundlich und gegen Fehleingaben geschützt.

Freigabeurteil: Das System wird hiermit uneingeschränkt für den Produktivbetrieb im Gastronomiebetrieb freigegeben.